**Dokumentation der Programmieraufgaben**

**1. Aufgabe 5: Bisektionsverfahren (Intervallhalbierung)**

**Mathematischer Hintergrund**

Wie in meinem Begleitdokument **"mathematischeherleitungen.pdf" (Seite 1)** beschrieben, basiert dieses Verfahren auf dem Nullstellensatz von Bolzano. Wir formen ein Problem (z.B.

) in die Form um.

**Code-Erklärung**

- **Logik:** Der Kern des Programms ist die while-Schleife. Sie halbiert das Intervall so lange, bis die Breite des Intervalls kleiner als die gewünschte Genauigkeit (eps) ist.
- **Entscheidung:** Durch die Prüfung if f(a) * f(c) < 0 bestimmen wir, ob die Nullstelle in der linken oder rechten Hälfte liegt.
- **Flexibilität:** Dank der eval()-Funktion kann der Nutzer mathematische Ausdrücke direkt als Text eingeben.

```
Funktion f(x) (z.B. x**2 - 3): x**2 - 3
Intervallgrenze a: 1
Intervallgrenze b: 2
Genauigkeit (z.B. 0.001): 0.001

------------------------------------------------------------
It.  | a           | b           | Mitte c     | f(c)
------------------------------------------------------------
1    | 1.000000    | 2.000000    | 1.500000    | -0.750000
2    | 1.500000    | 2.000000    | 1.750000    | 0.062500
3    | 1.500000    | 1.750000    | 1.625000    | -0.359375
4    | 1.625000    | 1.750000    | 1.687500    | -0.152344
5    | 1.687500    | 1.750000    | 1.718750    | -0.045898
6    | 1.718750    | 1.750000    | 1.734375    | 0.008057
7    | 1.718750    | 1.734375    | 1.726562    | -0.018982
8    | 1.726562    | 1.734375    | 1.730469    | -0.005478
9    | 1.730469    | 1.734375    | 1.732422    | 0.001286
------------------------------------------------------------
Ergebnis: 1.731445 nach 9 Iterationen.
```

---

**2. Aufgabe 6: Regula Falsi (Sekantenverfahren)**

**Mathematischer Hintergrund**

Die Herleitung der Formel über die Punkt-Steigungs-Form der Sekante findet sich in den **Herleitungen auf Seite 2**. Im Gegensatz zur Bisektion wird hier kein Mittelpunkt, sondern ein gewichteter Schnittpunkt berechnet.

**Code-Erklärung**

- **Berechnung:** Die Zeile c = (a * fb - b * fa) / (fb - fa) implementiert die Sekantenformel.

- **Abbruchbedingung:** Hier wird nicht die Intervallbreite, sondern der Funktionswert abs(f(c)) geprüft. Ist dieser nahe genug an Null, bricht das Programm ab.

```
Funktion f(x): x**2 - 5
Grenze a: 2
Grenze b: 3
Genauigkeit: 0.001

------------------------------------------------------------
It.  | a            | b            | Schnitt c    | f(c)
------------------------------------------------------------
1    | 2.000000     | 3.000000     | 2.200000     | -0.160000
2    | 2.200000     | 3.000000     | 2.230769     | -0.023669
3    | 2.230769     | 3.000000     | 2.235294     | -0.003460
4    | 2.235294     | 3.000000     | 2.235955     | -0.000505
------------------------------------------------------------
Ergebnis: 2.235955 nach 4 Iterationen.
```

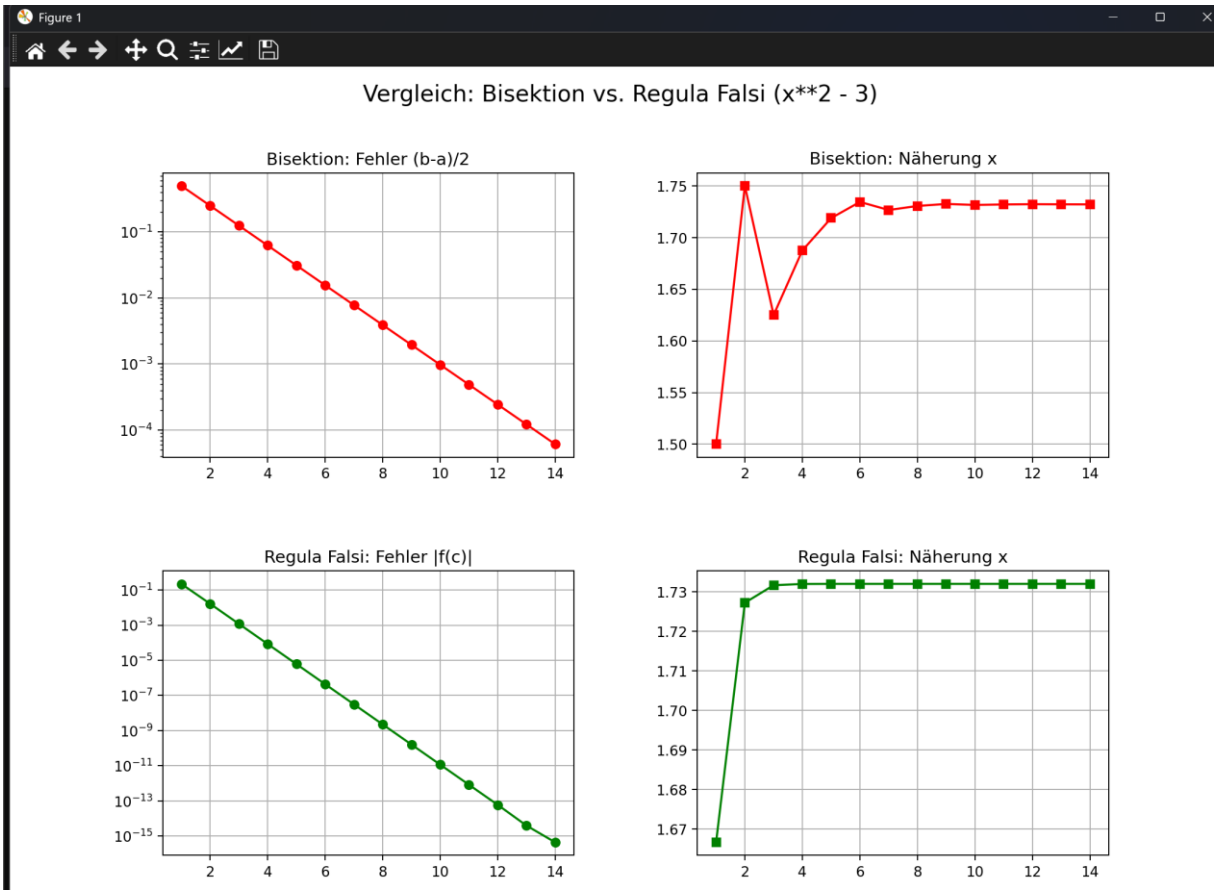
---

### 3. Aufgabe 7: Visualisierung

**Beschreibung**

Um die Lage der Nullstellen besser zu verstehen, wird die Funktion grafisch dargestellt.

- **Bibliotheken:** Wir nutzen numpy für die Wertebereiche und matplotlib für die Zeichnung.
- **Nulllinie:** Mit plt.axhline(0) wird die x-Achse markiert, um die Schnittpunkte (Nullstellen) sofort sichtbar zu machen.




---

### 4. Aufgabe 8: Lösung des Theorie-Polynoms

#### Zielsetzung

In dieser Aufgabe wenden wir den Bisektions-Algorithmus auf das spezifische Polynom an.

- **Durchführung:** Das Programm ist fest auf dieses Polynom eingestellt und testet das Intervall .
- **Ergebnis:** Das Programm bestätigt die theoretische Nullstelle bei .

```
--- AUFGABE 8: Theorie-Polynom P4(x) ---
Berechnete Nullstelle für P4(x) im Intervall [1.5, 2.5]: 2.0
PS C:\Users\letsl\OneDrive - tgm - Die Schule der Technik\3BHWII\swp\Git>
```

---

### 5. Aufgabe 9: Anwendung Elektrische Leitung & Integration

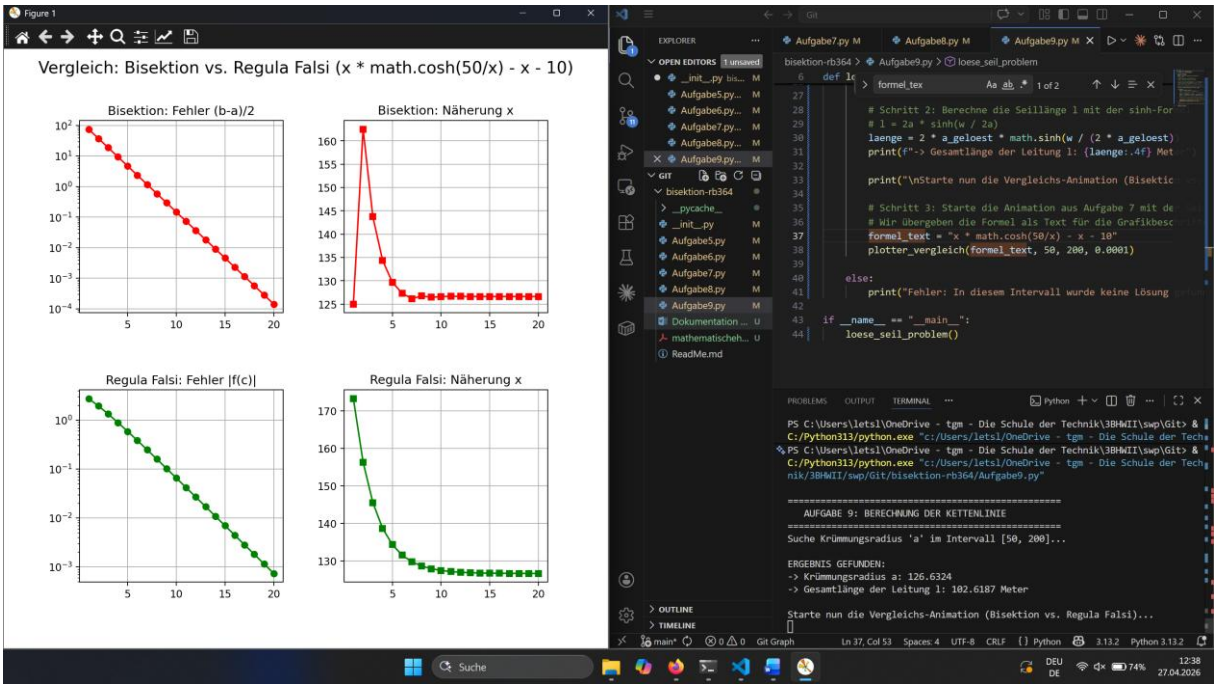
#### Mathematischer Hintergrund

Die Berechnung der Kettenlinie (Durchhang eines Seils) mittels des Kosinus Hyperbolicus () ist auf **Seite 3 der Herleitungen** mathematisch aufbereitet.

#### Code-Erklärung & Struktur

- **Integration:** Diese Datei dient als Hauptprogramm. Sie importiert die Funktionen solver, solver2 und plotter aus den anderen Dateien.

- **Einzeltestbarkeit:** Durch die Verwendung von if __name__ == "__main__": in allen Dateien wird sichergestellt, dass jede Aufgabe einzeln gestartet werden kann, ohne dass der Code der anderen Aufgaben ungewollt ausgeführt wird. Dies erfüllt die Anforderung der "Einzel-Testbarkeit" ohne manuellen Eingriff in den Code.




# Detaliertere Code erklärung

---

**Aufgabe 5: Bisektionsverfahren (Intervallhalbierung)**

In diesem Code wurde das klassische Bisektionsverfahren implementiert.

- **Funktionsweise:** Das Programm teilt ein gegebenes Intervall [a, b] immer exakt in der Mitte (c). Es prüft durch einen Vorzeichenwechsel, in welcher Hälfte die Nullstelle liegen muss, und verkleinert das Suchfeld Schritt für Schritt.
- **Besonderheit:** Der Code gibt eine strukturierte Tabelle im Terminal aus, die jeden Iterationsschritt mit den aktuellen Grenzen und dem Funktionswert dokumentiert. Dies dient der Verifizierung der linearen Konvergenz.

**Aufgabe 6: Regula Falsi (Sekantenverfahren)**

Dieses Programm nutzt die Regula Falsi zur Nullstellenbestimmung.

- **Funktionsweise:** Im Gegensatz zur Bisektion wird hier keine einfache Mitte gewählt. Stattdessen wird eine Gerade (Sekante) zwischen den Funktionswerten an den Intervallgrenzen gezogen. Der Schnittpunkt dieser Geraden mit der x-Achse ist die neue Näherung.

- **Ergebnis:** Dieses Verfahren gewichtet die Grenzen nach ihren Funktionswerten, was in vielen Fällen zu einer deutlich schnelleren Annäherung führt als die einfache Bisektion.

**Aufgabe 7: Grafische Visualisierung & Animation**

Hier wurde die mathematische Berechnung grafisch aufbereitet.

- **Technische Umsetzung:** Unter Verwendung von Matplotlib Subplots wurde ein Layout mit vier Diagrammen erstellt.
- **Animation:** Durch die Pause-Funktion im Code wird die Berechnung live animiert. Man sieht, wie die Fehlerkurven nach unten sinken (Genauigkeit) und wie sich die Näherungswerte stabilisieren.
- **Vergleich:** Die Gegenüberstellung zeigt direkt, welches Verfahren für eine bestimmte Funktion effizienter arbeitet.

**Aufgabe 8: Nullstellenbestimmung bei Polynomen höherer Ordnung**

In dieser Aufgabe wurde die Robustheit der Algorithmen an einem komplexeren Polynom (P4) getestet.

- **Ziel:** Es sollte nachgewiesen werden, dass das Programm auch bei Funktionen mit mehreren Krümmungen die exakte Nullstelle (hier $x = 2{,}0$) innerhalb der vorgegebenen Toleranz findet. Dies validiert die Logik für allgemeinere mathematische Probleme.

**Aufgabe 9: Praxisanwendung – Die Kettenlinie**

Die letzte Aufgabe kombiniert alle vorherigen Bausteine für ein physikalisches Problem (Stromleitung zwischen zwei Masten).

- **Problemstellung:** Die Berechnung des Krümmungsradius "a" einer hängenden Kette. Die zugrunde liegende Gleichung ist transzendent und kann nur numerisch gelöst werden.
- **Modularität:** Das Programm importiert die Logik aus Aufgabe 5 und den Plotter aus Aufgabe 7.
- **Ergebnis:** Nach der numerischen Bestimmung von "a" (ca. 126,63) wird über den Sinus Hyperbolicus (sinh) die exakte Seillänge von ca. 102,62 Metern berechnet.